

SOC DETECTION & INCIDENT INVESTIGATION

A Wazuh Portfolio Case Study

PORTFOLIO FOCUS Detection engineering, alert triage, event correlation, vulnerability remediation, security configuration assessment, and false-positive analysis.

LAB	Windows 11 endpoint + Wazuh 4.14.7 single-node Docker deployment
ENDPOINT	LAB-WIN11 Agent 001 Windows Security + Sysmon + FIM
ROLE	Independent analyst and detection engineer
DATE	September 2026

Prepared as an employer-ready demonstration for entry-level SOC and defensive-security roles.

Executive Summary

This project built and operated a functional endpoint-monitoring lab, then used it to complete realistic SOC investigations from raw telemetry through analyst verdict. The environment collected Windows Security, Sysmon, and file-integrity data; generated and validated detections; correlated identity, process, and network events; remediated a vulnerable application and a failed CIS control; and closed with an independent false-positive investigation.

PUBLIC RELEASE NOTE Personal identifiers in the narrative and screenshots were consistently pseudonymized. LAB-WIN11 and AnalystUser represent the original lab endpoint and analyst account; private IPv4 and machine SID components are masked.

RESULT The lab produced defensible evidence across twelve completed tasks. Findings were separated into directly observed facts, Wazuh classifications, analytical inferences, unresolved uncertainty, and final disposition.

Environment and Data Sources

Component	Implementation	Analytical purpose
Endpoint	Windows 11 Pro, LAB-WIN11	Security, process, file, identity, and network telemetry
Wazuh	4.14.7, Docker single-node	Collection, rules, alerting, threat hunting, vulnerability/SCA
Agent	001, active	Windows event and inventory forwarding
Sysmon	Sysmon64, Operational log	Event 1 process creation; Event 3 network; Event 11 file creation
Rule file	/var/ossec/etc/rules/local_rules.xml	Authoritative custom detection location

Skills Demonstrated

- Alert triage and evidence preservation
- Windows Security and Sysmon investigation
- Detection-rule troubleshooting and boundary testing
- Event correlation across identity, process, and network telemetry
- MITRE ATT&CK interpretation without overstating intent
- Vulnerability remediation and control validation
- False-positive analysis and targeted tuning recommendations

Investigation Portfolio

The first eight tasks established the telemetry baseline and repeated a consistent analyst workflow: define the objective, trigger controlled activity, query the correct data source, validate the underlying event, interpret rule output, and document a cautious verdict.

Task	Evidence and rule	Analyst outcome
FIM lifecycle	Creation, modification, deletion; checksum/attribute changes	Validated file-integrity visibility and differentiated lifecycle actions
PowerShell execution	Sysmon Event 1; Wazuh 92027	Recovered command line, parent, user, integrity, and process identity
Failed authentication	Security Event 4625; Wazuh 60122	Interpreted account, logon type, status, and failure context
Account lifecycle	4720, 4722, 4738, 4726	Tracked actor, target account, SID, creation/change/deletion
Network telemetry	Sysmon Event 3; 61605 → 92101	Separated endpoint event, level-0 classifiers, and visible alert

ANALYTICAL STANDARD A MITRE mapping or high alert level was treated as behavioral context—not proof of malicious intent.

Detection Engineering: PowerShell TCP Alert

Objective and Rule Logic

The goal was to promote Wazuh's built-in, level-0 PowerShell-over-TCP classification into an analyst-visible alert while preserving the native rule hierarchy. Earlier attempts inherited from the generic Event 3 classifier or matched overly broad conditions; those failures were retained as troubleshooting evidence but excluded from the final control.

Layer	Role	Visibility
61605	Generic Sysmon Event 3 classifier	Level 0
92101	PowerShell communicating over TCP	Level 0
100100	Custom promotion of the specific classification	Level 8

Figure 1. Final authoritative custom rule in local_rules.xml. The description uses “observed” because parent rule 92101 does not guarantee initiated:true.

< local_rules.xml
Ruleset Test
Save

```

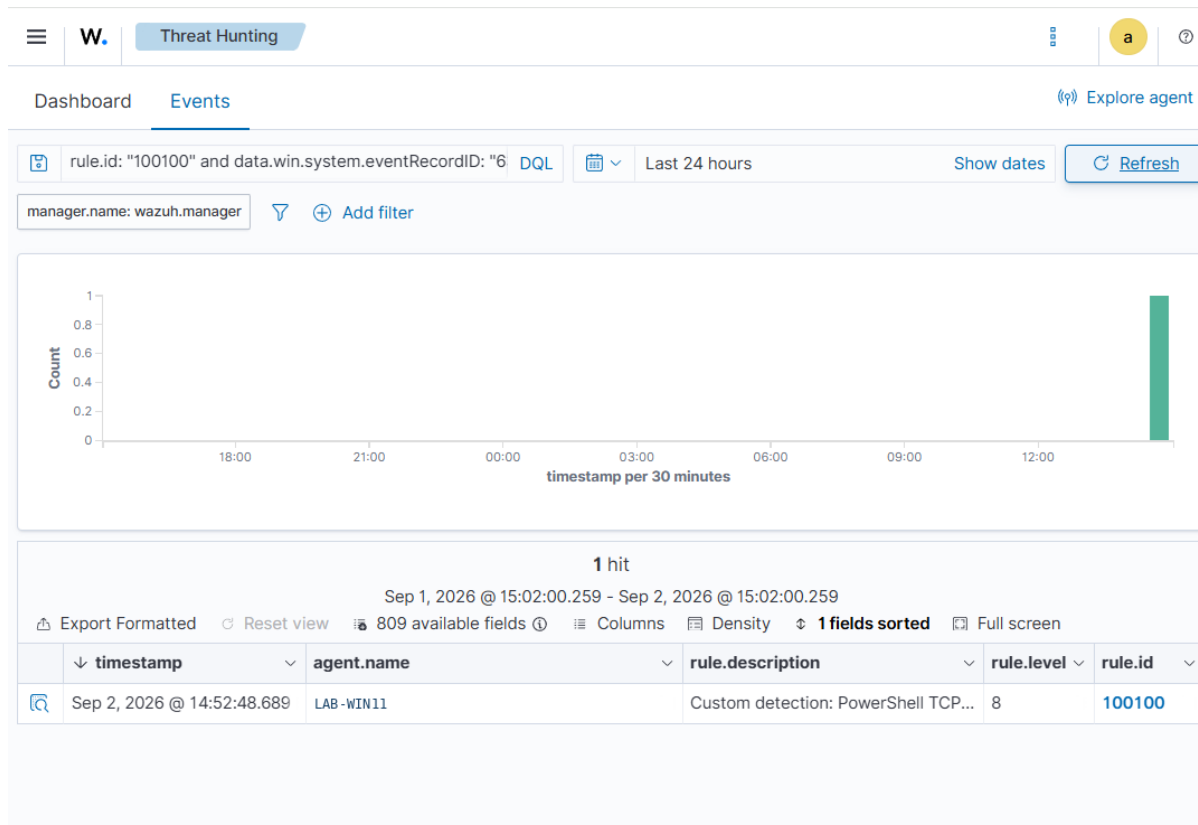
1 <!-- Custom PowerShell network detection -->
2 <group name="local, windows, sysmon, network, ">
3   <rule id="100100" level="8">
4     <if_sid>92101</if_sid>
5     <description>Custom detection: PowerShell TCP connection observed to $(win.eventdata.destinationIp):$(
6       (win.eventdata.destinationPort)</description>
7     <mitre>
8       <id>T1059.001</id>
9       <id>T1095</id>
10    </mitre>
11    <group>custom_detection, powershell_network, </group>
12  </rule>
13 </group>

```

Validation and Boundary Test

A controlled PowerShell Test-NetConnection to 1.1.1.1:443 produced rule 100100 at level 8. The final alert showed PowerShell, TCP, initiated:true, user AnalystUser, and the external destination. An equivalent non-PowerShell connection from Code.exe was archived as Sysmon Event 3 but generated zero rule-100100 alerts, confirming the process boundary.

Figure 2. Wazuh dashboard validation for custom rule 100100.



VERDICT Authorized benign positive. T1059.001 is directly supported by the PowerShell process. T1095 is inherited behavioral context and was not treated as proof of command-and-control.

Production Tuning Recommendation

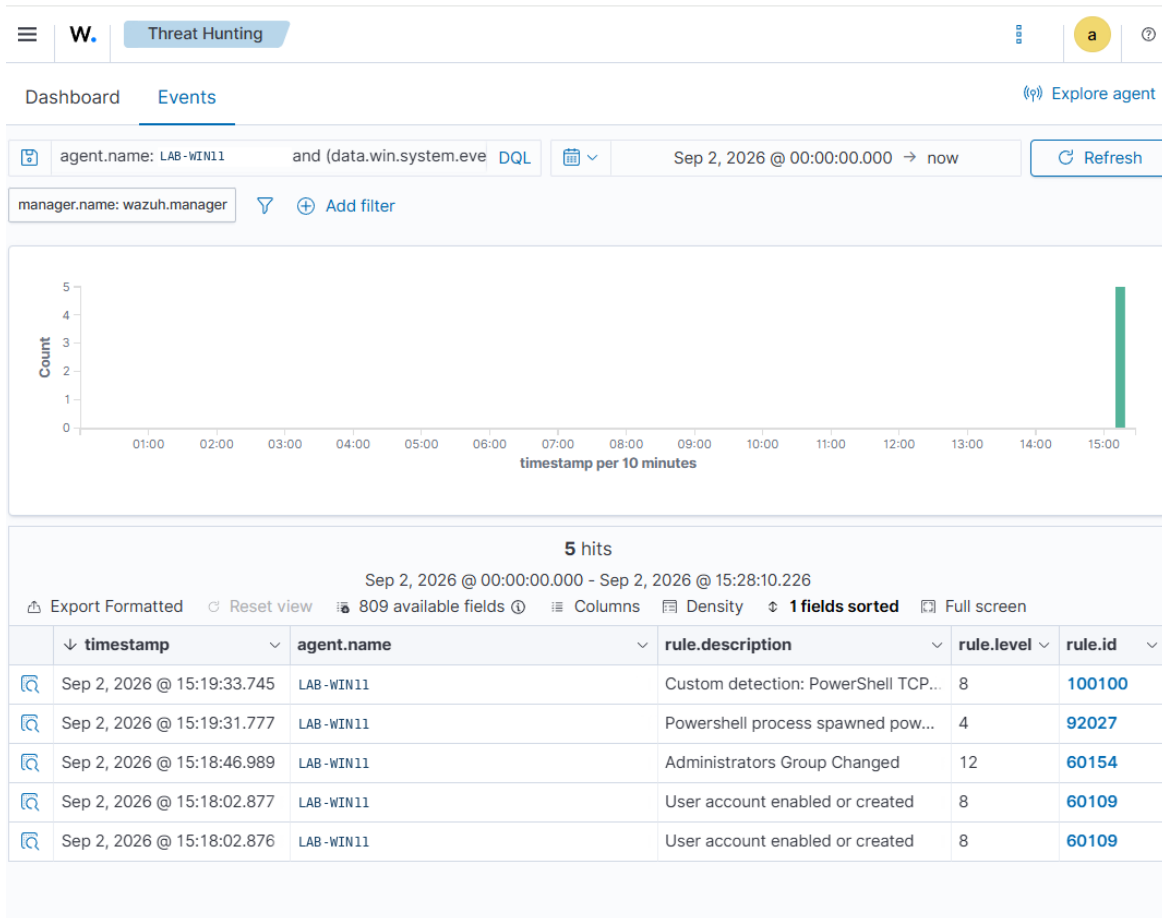
- Baseline trusted automation, accounts, and destinations.
- Raise severity only when supported by suspicious command lines, encoded content, unusual parent processes, rare destinations, or repeated connections.
- Retain the high-sensitivity rule for hunting; consider a lower default severity for uncorrelated administrative activity.

Correlated Suspicious Sequence

A controlled sequence linked account creation, enablement, privileged-group membership, PowerShell execution, and outbound TCP activity. Correlation used stable identity and process fields rather than timestamp proximity alone.

Time	Event / rule	Observed activity
15:18:02	4720 / 60109	SOC-SequenceUser created
15:18:02	4722 / 60109	SOC-SequenceUser enabled
15:18:46	4732 / 60154	Account added to local Administrators
15:19:31	Sysmon 1 / 92027	PowerShell process created
15:19:33	Sysmon 3 / 100100	PowerShell TCP to 1.1.1.1:443

Figure 3. Threat-hunting timeline for the five-stage controlled sequence.



Correlation Keys

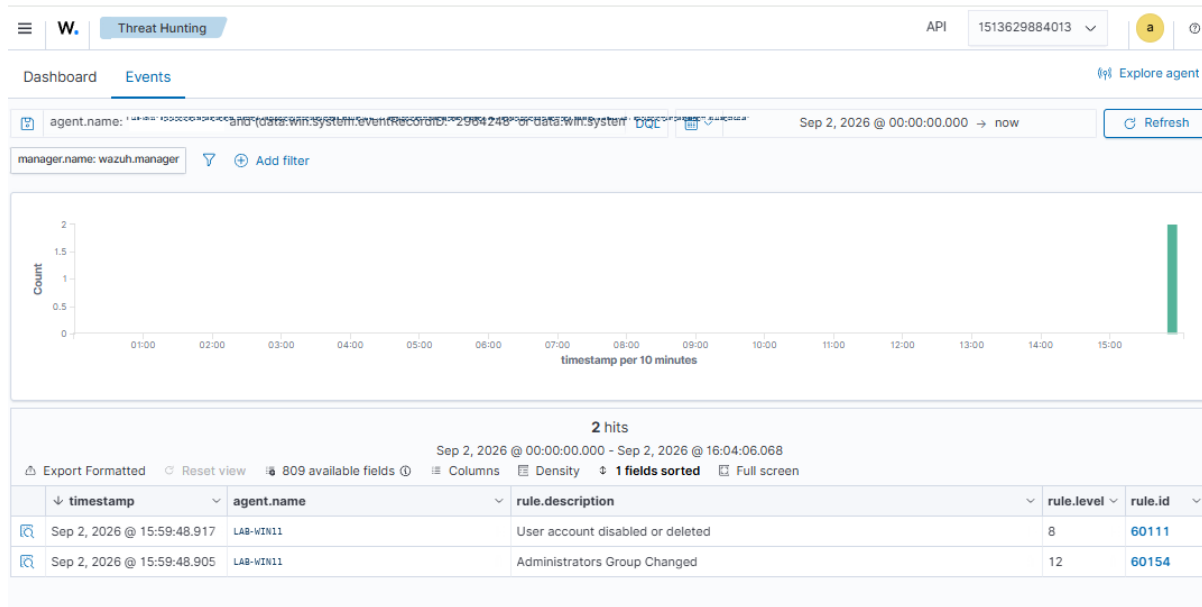
- Actor: LAB-WIN11\AnalystUser and Logon ID 0x1F0370 tied the account and process actions to the same administrative session.
- Target SID S-1-5-21-****-****-****-1005 identified SOC-SequenceUser across account and group events.
- Process GUID {a8b37afb-76c2-6a98-1cf7-010000005b00} and PID 55520 linked the PowerShell process to its network event.
- Network tuple: 192.168.1.xxx:57892 → 1.1.1.1:443.

VERDICT Authorized simulation with a suspicious pattern. The sequence resembles account-based persistence followed by privileged execution and external communication, but the lab context and controlled trigger prevent a malicious conclusion.

Cleanup Verification

The account was removed from Administrators (Event 4733, Record 2964248) and deleted (Event 4726, Record 2964250). Local checks confirmed StillExists=False and StillAdministrator=False.

Figure 4. Wazuh cleanup alerts for group-member removal and account deletion.



Vulnerability and Configuration Remediation

Docker Desktop Finding

Wazuh identified CVE-2026-6406 against Docker Desktop 4.54.0. The finding was rated High with CVSS 8.8 and indicated affected versions below 4.59.0. Independent file-version validation confirmed 4.54.0.212467. The user AnalystUser was a member of docker-users, while Enhanced Container Isolation status remained unconfirmed; no exploitation evidence was observed.

Figure 5. Vulnerability details for CVE-2026-6406, including package version, severity, score, and affected-version condition.

Vulnerability details

Table JSON

_index	wazuh-states-vulnerabilities-wazuh.manager
agent.id	001
agent.name	TEM_MACHINE
agent.type	Wazuh
agent.version	v4.14.7
host.os.full	Microsoft Windows 11 Pro 10.0.26200.9168
host.os.kernel	10.0.26200.9168
host.os.name	Microsoft Windows 11 Pro
host.os.platform	windows
host.os.type	windows
host.os.version	10.0.26200.9168
package.architecture	x86_64
package.installed	Dec 9, 2025 @ 00:43:03.000
package.name	Docker Desktop
package.path	C:\Program Files\Docker\Docker
package.size	0
package.type	win
package.version	4.54.0
vulnerability.category	Packages
vulnerability.classification	CVSS
vulnerability.description	If the host user has logged in to container registries, their authentication credentials. A local attacker with the ability to run Docker CLI commands can exploit this to escape ECI restrictions, access the Docker Engine, and potentially escalate privileges.
vulnerability.detected_at	Sep 2, 2026 @ 16:05:23.000
vulnerability.enumeration	CVE
vulnerability.id	CVE-2026-6406
vulnerability.published_at	May 22, 2026 @ 15:17:04.000
vulnerability.reference	https://docs.docker.com/desktop/release-notes/#4590 , https://www.zerodayinitiative.com/advisories/ZDI-26-299/
vulnerability.scanner.condition	Package less than 4.59.0
vulnerability.scanner.reference	https://cti.wazuh.com/vulnerabilities/cves/CVE-2026-6406
vulnerability.scanner.source	National Vulnerability Database
vulnerability.scanner.vendor	Wazuh
vulnerability.score.base	8.8
vulnerability.score.version	3.1
vulnerability.severity	High
vulnerability.under_evaluation	false
wazuh.cluster.name	wazuh.manager
wazuh.schema.version	1.0.0

Remediation and Validation

Docker Desktop was upgraded to 4.89.0.238018. The Wazuh dashboard, manager, and indexer containers returned to a running state, and agent 001 remained active. After inventory refresh and rescan, an exact CVE query returned no results.

Figure 6. Post-remediation Docker version and Wazuh health validation.

```

PS C:\WINDOWS\system32> $DockerDesktop = Get-Item `
>> "C:\Program Files\Docker\Docker\Desktop.exe"
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> [PSCustomObject]@{
>>   ProductVersion = $DockerDesktop.VersionInfo.ProductVersion
>>   FileVersion    = $DockerDesktop.VersionInfo.FileVersion
>> }

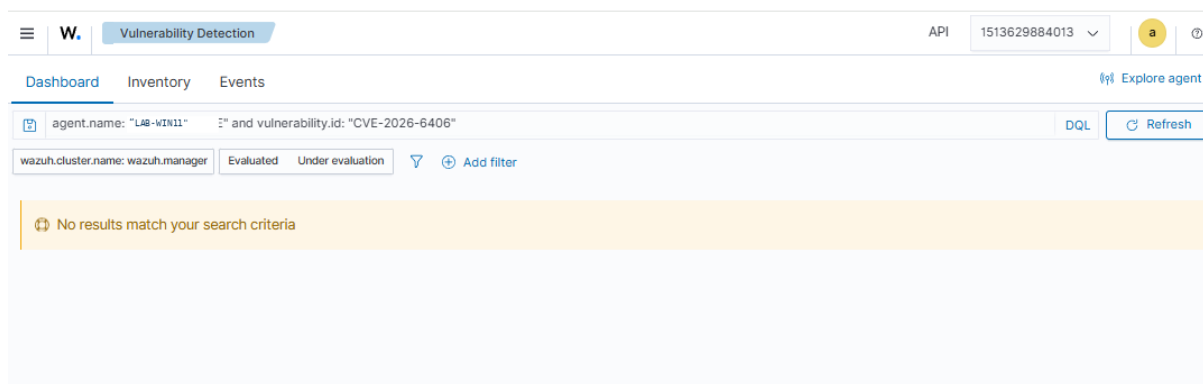
ProductVersion FileVersion
-----
4.89.0.238018 4.89.0.238018

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> docker ps `
>> --filter "name=single-node-wazuh" `
>> --format "table {{.Names}}\t{{.Status}}"
NAMES          STATUS
single-node-wazuh.dashboard-1 Up 4 minutes
single-node-wazuh.manager-1 Up 4 minutes
single-node-wazuh.indexer-1 Up 4 minutes
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> docker exec single-node-wazuh.manager-1 `
>> /var/ossec/bin/agent_control -lc

Wazuh agent_control. List of available agents:
ID: 000, Name: wazuh.manager (server), IP: 127.0.0.1, Active/Local
ID: 001, Name: LAB-WIN11, IP: any, Active

```

Figure 7. Exact CVE query showing the finding cleared after remediation.



VERDICT Remediated and validated. The dashboard finding established exposure by version; it did not establish exploitability or compromise.

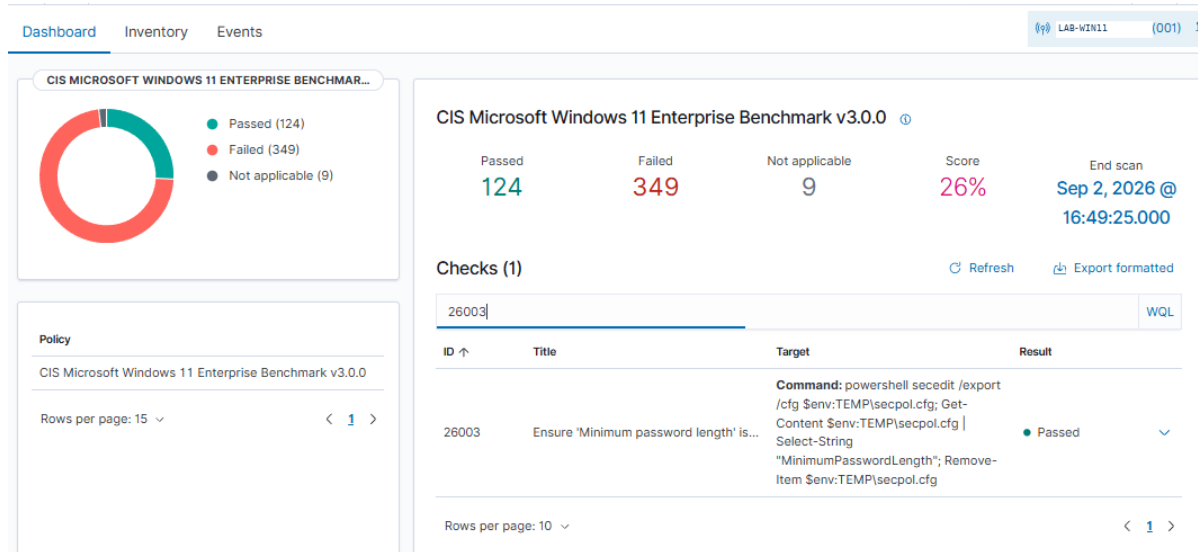
Security Configuration Assessment

The CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0 scan initially reported 123 passed, 350 failed, and 9 not applicable, with a rounded score of 26%. Because the host runs Windows 11 Pro, the Enterprise benchmark was treated as a useful control reference with an applicability caveat—not an exact certification statement.

Selected Control: Minimum Password Length

Check 26003 required a minimum password length of at least 14 characters. The native net accounts output showed a value of 0, confirming a true positive. After exporting a security-policy backup, the minimum length was set to 14 and Wazuh rescanned the endpoint.

Figure 8. SCA control 26003 passed after remediation; totals changed to 124 passed and 349 failed.



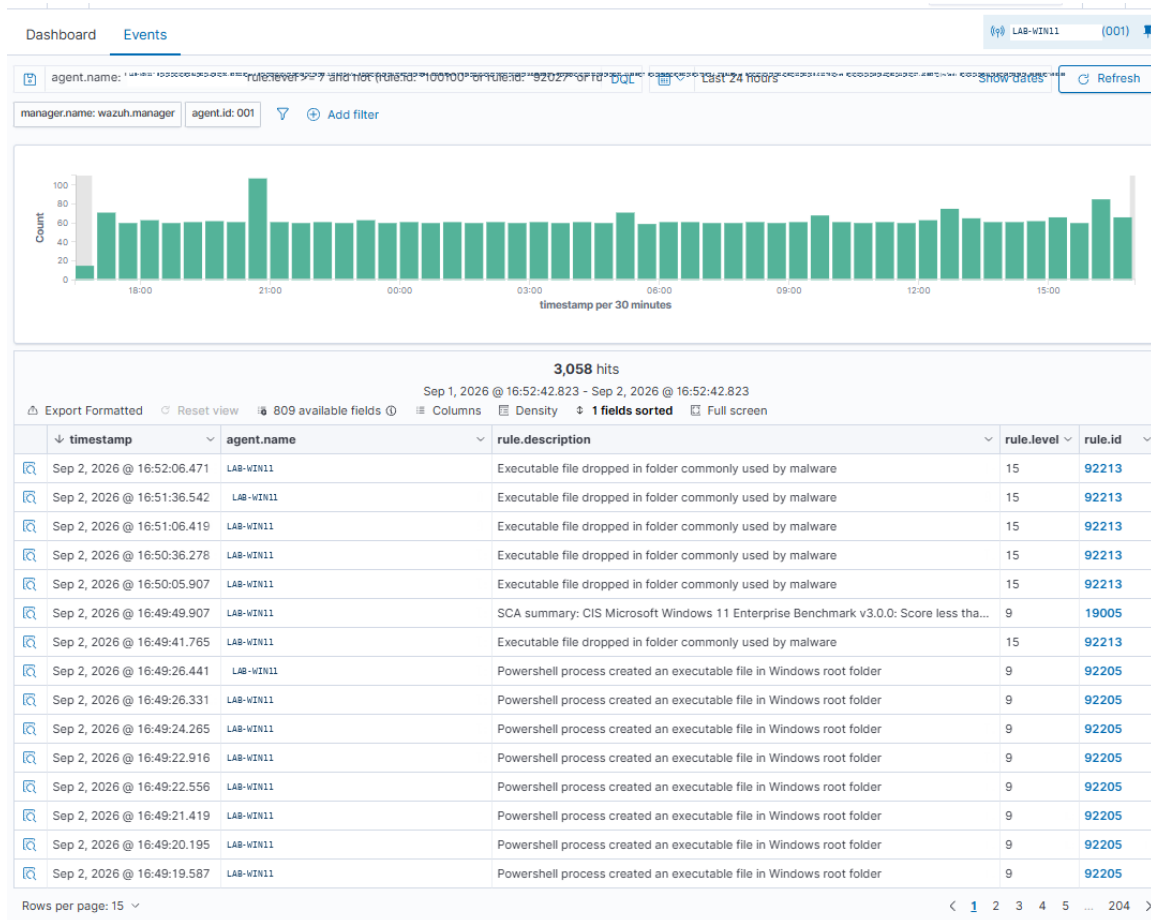
INTERPRETATION The displayed score remained 26% because it is rounded. One control moved from failed to passed; the benchmark score is not a general “security percentage.”

Independent Investigation: Rule 92213

Triage Selection

From a queue of 3,058 high-severity alerts, a level-15 event labeled “Executable file dropped in folder commonly used by malware” was selected for independent analysis. The event involved a temporary PowerShell policy-test script and therefore required validation of the creating process, parent lineage, related behavior, and rule logic.

Figure 9. High-severity alert queue used to select the independent investigation.



Observed Evidence

Field	Value
Alert	Rule 92213, level 15; Sysmon Event 11, Record 688407
Target	C:\Users\AnalystUser\AppData\Local\Temp__PSScriptPolicyTest_lyb4r3a1.axm.ps1
Creator	C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe, PID 16484
Parent	C:\Program Files (x86)\ASUS\ArmouryDevice\asus_framework.exe, PID 25528
Parent trust	ASUSTeK valid signature; ASUS NodeJS Web Framework 4.2.4.13
Related activity	0 network connections; 0 child processes; 2 files created

Figure 10. Process provenance recovered from Wazuh archives after the local Sysmon event rolled over.

```

FoundInWazuhArchive : True
Timestamp           : 2026-09-02T20:52:06.311+0000
Agent               : LAB-WIN11
EventID             : 1
EventRecordID       : 688394
Image               : C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe
CommandLine         : powershell.exe -c gwm win32_process -Filter 'ProcessId=19788 or ProcessId=20120 or ProcessId=25300 or ProcessId=25760 or
                        ProcessId=26000 or ProcessId=25528' | select CreationDate,KernelModeTime,ParentProcessId,ProcessId,UserModeTime,WorkingSetSize |
                        format-table
User                : LAB-WIN11\timot
IntegrityLevel       : High
LogonID             : 0x1f0370
ProcessGUID         : {a8b37afb-8c75-6a98-5716-020000005b00}
ProcessID           : 16484
ParentImage         : C:\Program Files (x86)\ASUS\ArmouryDevice\asus_framework.exe
ParentProcessID     : 25528
ParentUser          : LAB-WIN11\timot

```

Rule Logic and Disposition

Rule 92213 matches any Sysmon Event 11 file written under a user Local Temp directory with an executable or script-like extension, including .ps1. It does not require evidence of download, execution, persistence, network activity, suspicious content, or an untrusted parent. The target file no longer existed at investigation time, and the same process created the expected StartupProfileData-NonInteractive file.

VERDICT High-confidence benign positive / operational false positive. The alert's path-and-extension heuristic fired correctly, but the surrounding evidence did not support ingress-tool transfer or malicious execution.

Targeted Tuning

Do not suppress all Temp-directory PowerShell scripts. A narrow exception should require the exact `__PSScriptPolicyTest_*.ps1` naming pattern, the expected Microsoft PowerShell image, and verified benign parent context. Deviations should remain visible.

Analyst Conclusions

The project demonstrates an end-to-end SOC workflow: deploy and validate telemetry, interpret native and custom rules, correlate activity across multiple sources, test detection boundaries, remediate exposed software and configuration, and investigate a severe alert to a defensible benign disposition. The strongest outcome is not simply that alerts fired; it is that each alert was tested against underlying evidence and uncertainty was documented explicitly.

Key Lessons

- Rule inheritance matters: the correct specific parent can be more important than adding more field conditions.
- Event presence, classification, and analyst-visible alerting are distinct layers.
- Correlation should rely on stable identifiers—SID, Logon ID, Process GUID, PID—not timestamps alone.
- Vulnerability inventory indicates exposure by package/version; it does not prove exploitability or compromise.
- High severity is a triage signal, not a verdict. Context can convert a level-15 alert into a well-supported false positive.
- Tuning should be narrow enough to preserve meaningful deviations.

Recommended Next Steps

- Version-control custom rules and document test cases.
- Add repeated negative tests for browser, editor, and approved automation traffic.
- Expand the sequence into a reusable detection-validation playbook.
- Track vulnerability and SCA remediation with before/after evidence and change records.
- Create a focused exception for the PowerShell policy-test artifact only after baseline validation.

Evidence Index

The case study uses representative screenshots in the narrative. The broader evidence set should remain organized separately as final evidence and troubleshooting evidence so that failed experiments remain auditable without being mistaken for the production rule.

Area	Representative evidence	Purpose
Custom detection	Figures 1–2	Final XML and dashboard validation
Correlation	Figures 3–4	Timeline and cleanup proof
Vulnerability	Figures 5–7	Finding, remediation health, and cleared query
SCA	Figure 8	Control passed after change
Independent investigation	Figures 9–10	Triage selection and archived provenance

EVIDENCE HYGIENE Troubleshooting screenshots—empty searches, incorrect rule variants, broad diagnostic alerts, restarts, and logtest output—are retained separately and clearly labeled. Command-entry errors and redundant duplicates are excluded from publication.